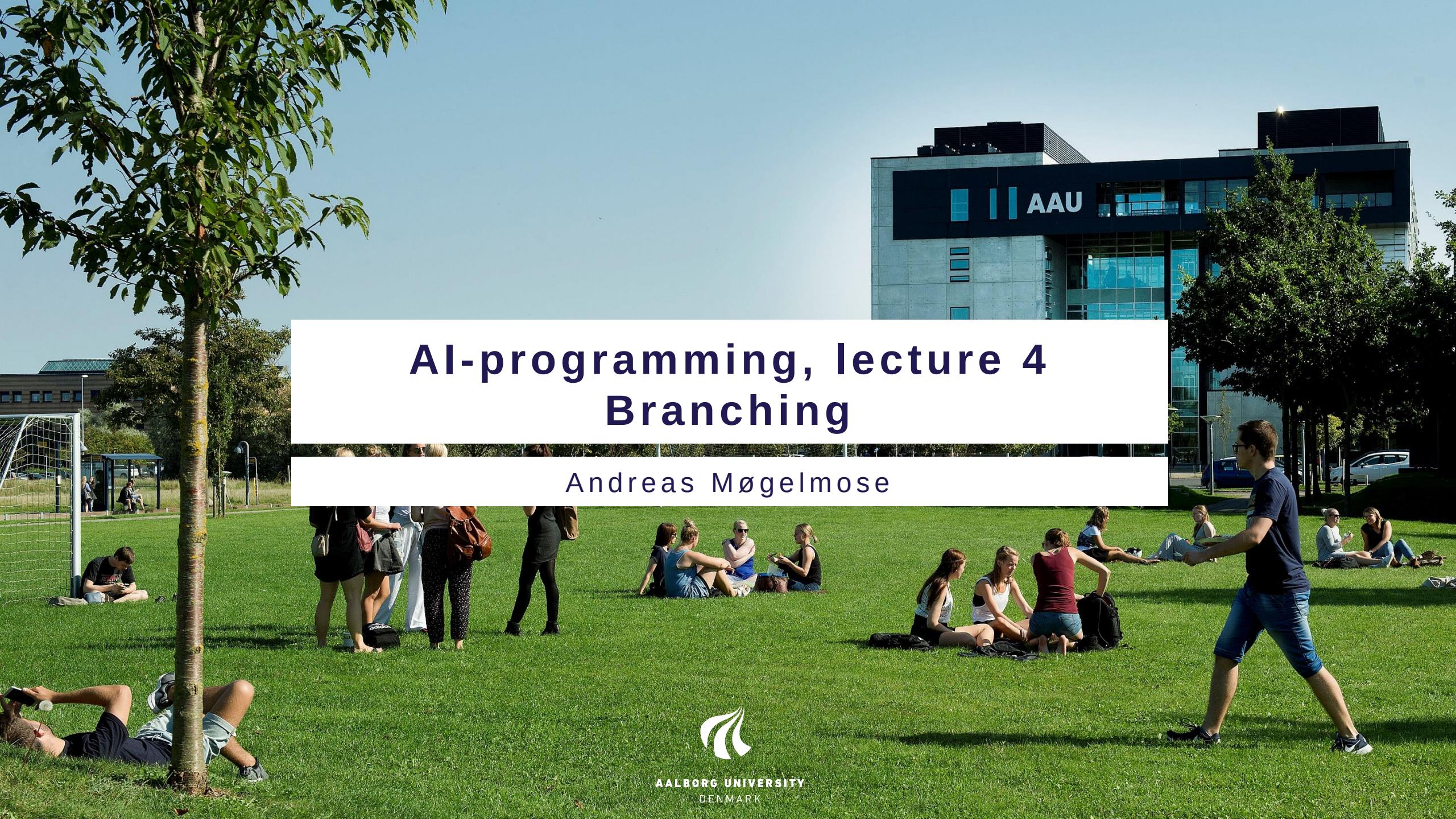


AI-programming, lecture 4 Branching

Andreas Møgelmose

AALBORG UNIVERSITY
DENMARK

Agenda

- Animation and a while sneak peek
- If statements
- Conditionals and boolean expressions
- Chaining and nesting



Recap: Functions

```
my_string.lower()

move(7, 2)

QuitProgram()
```



Recap: Variables and lists

```
name = 'Bob'
age = 21
print(f"My name is {name}. I am {age} years old.")

print(20 % 7)

vegetables = ['carrot', 'lettuce', 'onion', 'radish']
vegetables.append('potato')
```



Recap: Loops

```
cars = ['audi', 'bmw', 'subaru', 'toyota']
cars = [car.title() for car in cars]

for car in cars[1:3]:
    print(car)
```

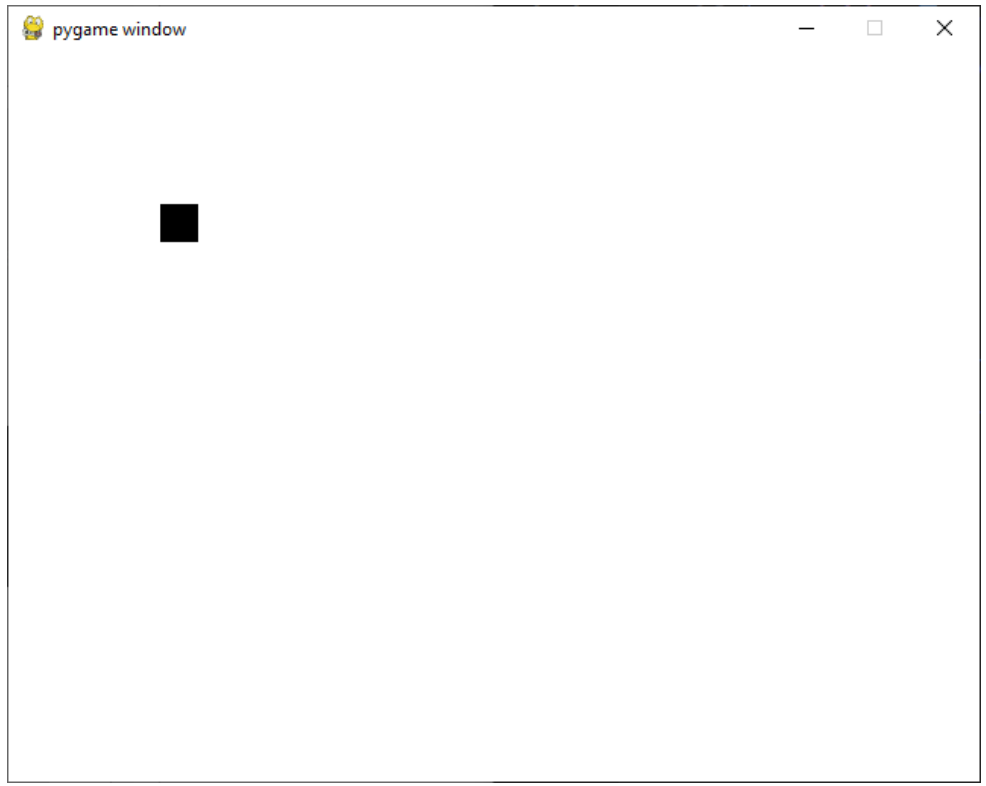


Agenda

- **Animation and a while sneak peek**
- If statements
- Conditionals and boolean expressions
- Chaining and nesting



Understanding the pygame boilerplate




```
import pygame
import math

pygame.init()
screen = pygame.display.set_mode((640, 480))
screen.fill((255, 255, 255))

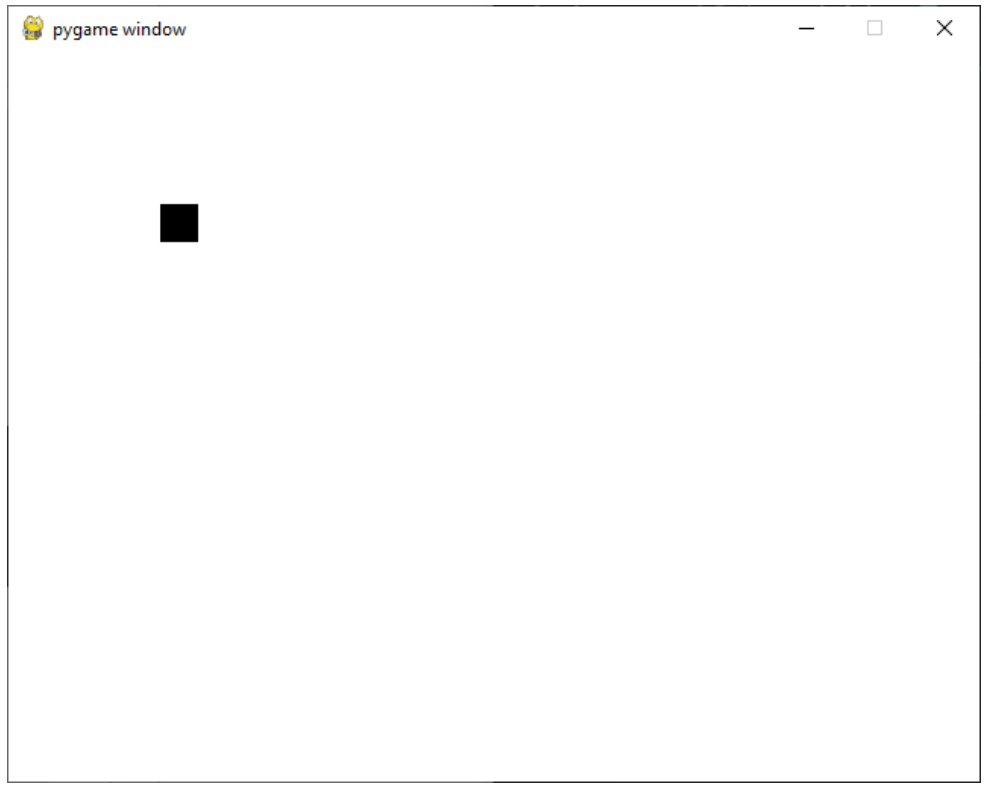
start_point = (100, 100)
box_size = (25, 25)

rect = start_point + box_size
pygame.draw.rect(screen, (0,0,0), rect)

pygame.display.flip()

while True:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            exit()
```

Animation




```
import pygame
import math

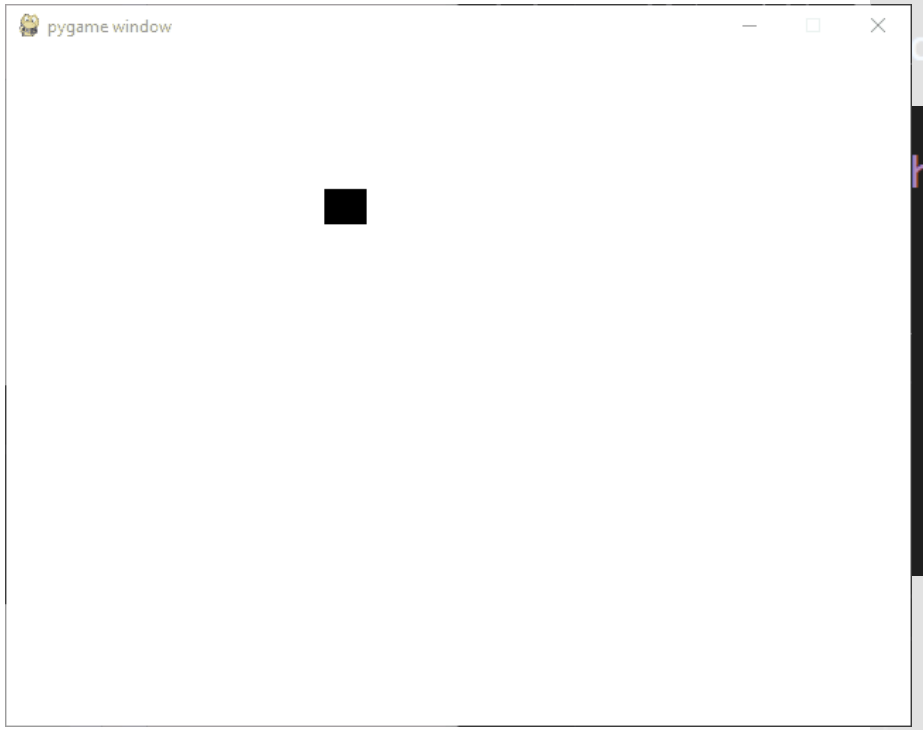
pygame.init()
screen = pygame.display.set_mode((640, 480))
screen.fill((255, 255, 255))

start_point = (100, 100)
box_size = (25, 25)

while True:
    rect = start_point + box_size
    pygame.draw.rect(screen, (0,0,0), rect)
    pygame.display.flip()

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            exit()
```

Animation




```
import pygame
import math
from datetime import datetime

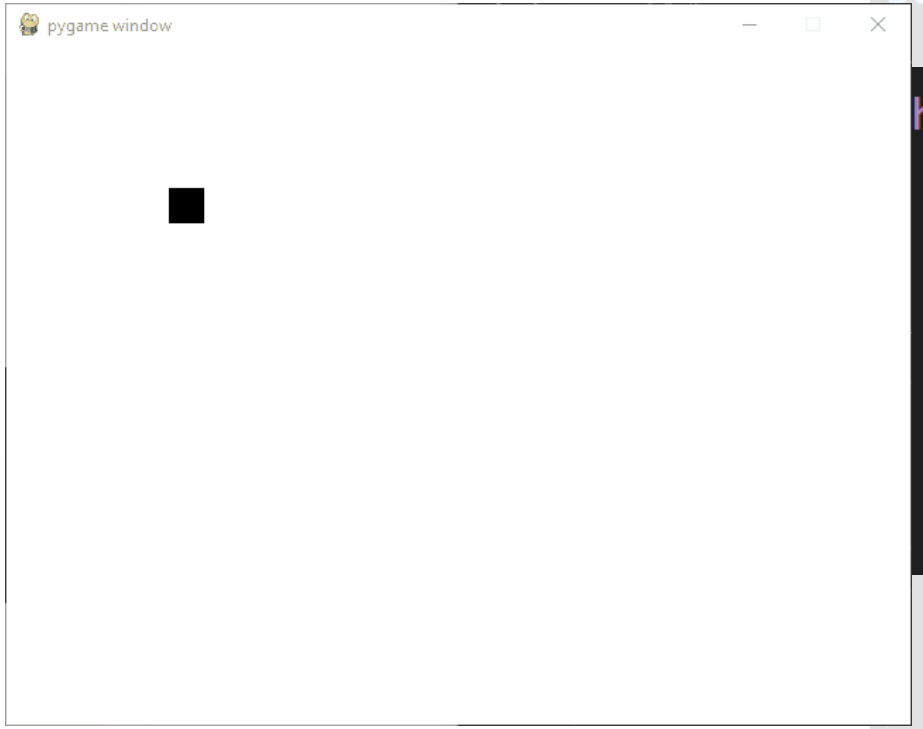
pygame.init()
screen = pygame.display.set_mode((640, 480))
screen.fill((255, 255, 255))

start_point = (100, 100)
box_size = (25, 25)

while True:
    box_position = (start_point[0] + datetime.now().second*5,
                    start_point[1])
    rect = box_position + box_size
    pygame.draw.rect(screen, (0,0,0), rect)
    pygame.display.flip()

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            exit()
```

Animation




```
import pygame
import math
from datetime import datetime

pygame.init()
screen = pygame.display.set_mode((640, 480))

start_point = (100, 100)
box_size = (25, 25)

while True:
    screen.fill((255, 255, 255))
    box_position = (start_point[0] + datetime.now().second*5,
                    start_point[1])
    rect = box_position + box_size
    pygame.draw.rect(screen, (0,0,0), rect)
    pygame.display.flip()

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            exit()
```

Agenda

- Animation and a while sneak peek
- **If statements**
- Conditionals and boolean expressions
- Chaining and nesting



Control flow

- What might happen apart from linear execution?
  - An unconditional jump to some other line of code
  - Jump back to repeat a block of code
  - Jump ahead, skipping a block of code based on some condition
  - Jump ahead, selecting one of several possible blocks of code to execute based on a condition
  - Unconditionally jump to some other block of code, only to return at a later point in time
  - Stop the program

What last lecture was about: Looping!

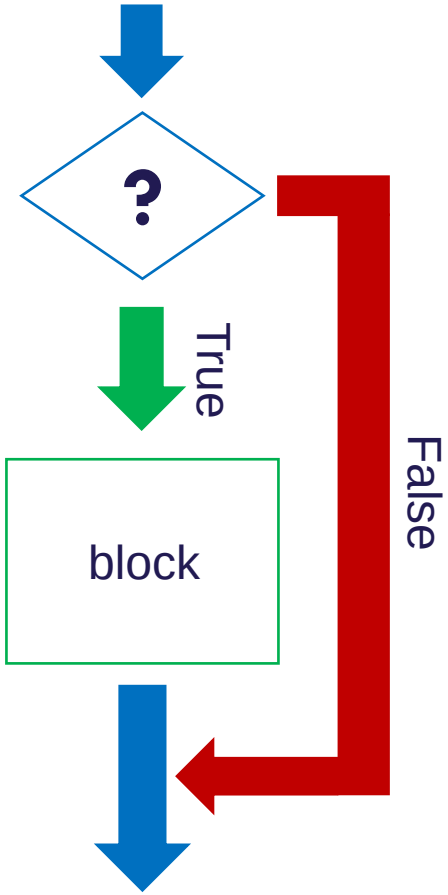
What this lecture is about: Branching!

What lecture 7 is about: Functions!



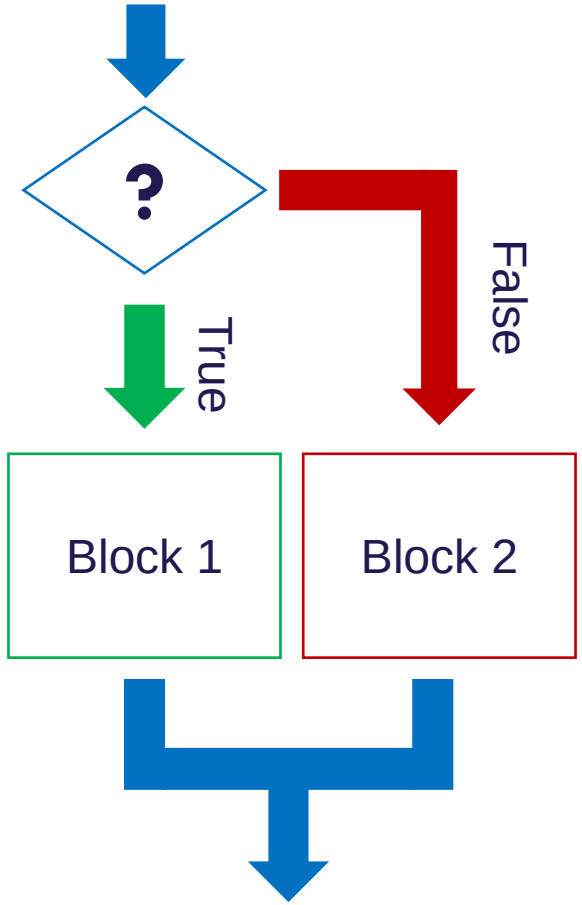
Control flow

- Skipping a block based on a condition



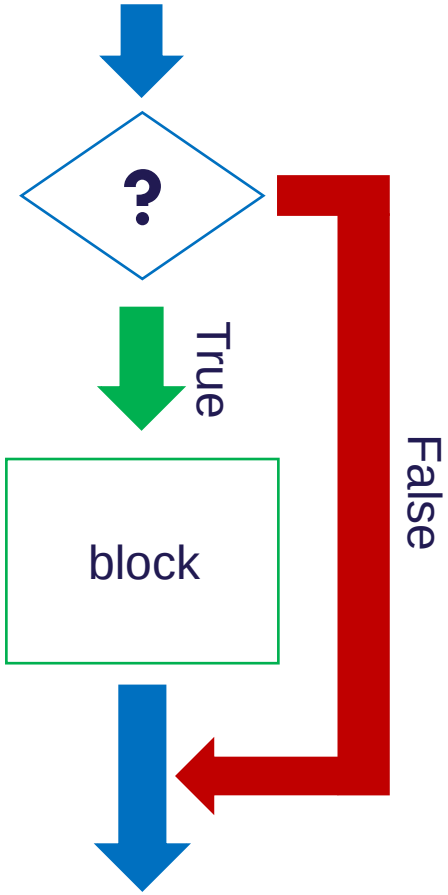
Control flow

- Selecting one of two blocks based on a condition



The `if` statement

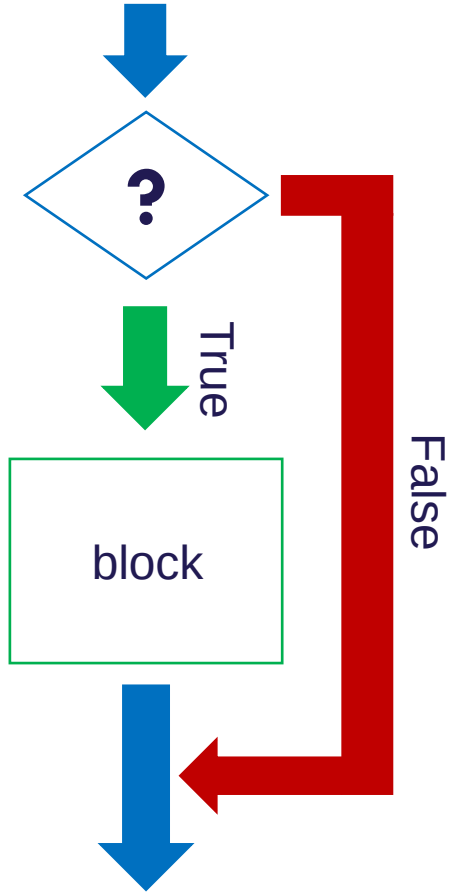
- How to skip a block of code?
  - Use an **`if`** statement
  - **`if`** is a reserved word in Python
  - The condition is essentially an expression with two possible values: `True` or `False`
  - Note that the flow merges again after the block



The `if` statement

- Syntax vs. illustration

```
code before
if condition:
    skippable code block
code after
```



If example

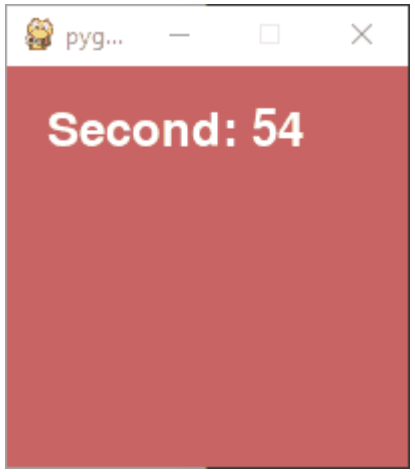
```
import pygame
import math
from datetime import datetime
pygame.init()
screen_size = (200, 200)
screen = pygame.display.set_mode(screen_size)
font = pygame.font.SysFont(None, 36)

while True:
    s = datetime.now().second
    screen.fill((100, 200, 100))
    if s > 30:
        screen.fill((200, 100, 100))

    text = font.render(f'Second: {s}', True, (255,
    screen.blit(text, (20, 20))

    pygame.display.flip()

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            exit()
```



If example

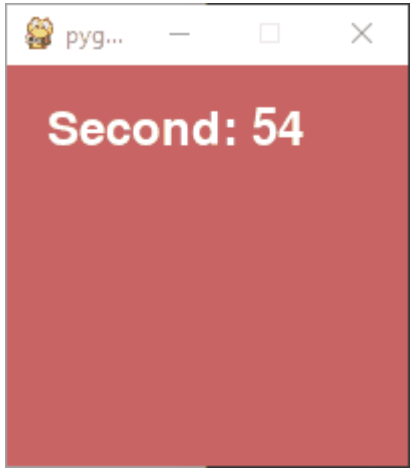
```
import pygame
import math
from datetime import datetime
pygame.init()
screen_size = (200, 200)
screen = pygame.display.set_mode(screen_size)
font = pygame.font.SysFont(None, 36)

while True:
    s = datetime.now().second
    screen.fill((100, 200, 100))
    if s > 30:
        screen.fill((200, 100, 100))

    text = font.render(f'Second: {s}', True, (255,
    screen.blit(text, (20, 20))

    pygame.display.flip()

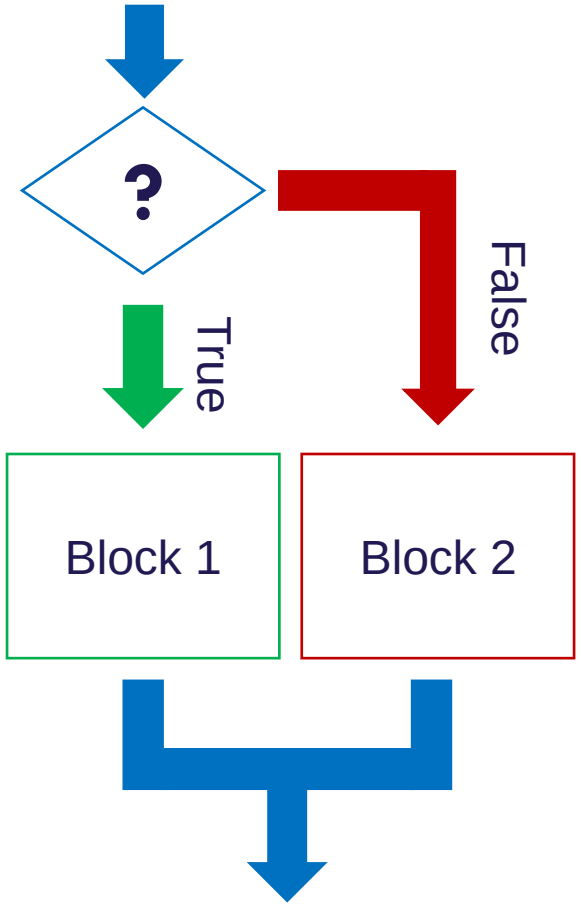
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            exit()
```



The `if-else` statement

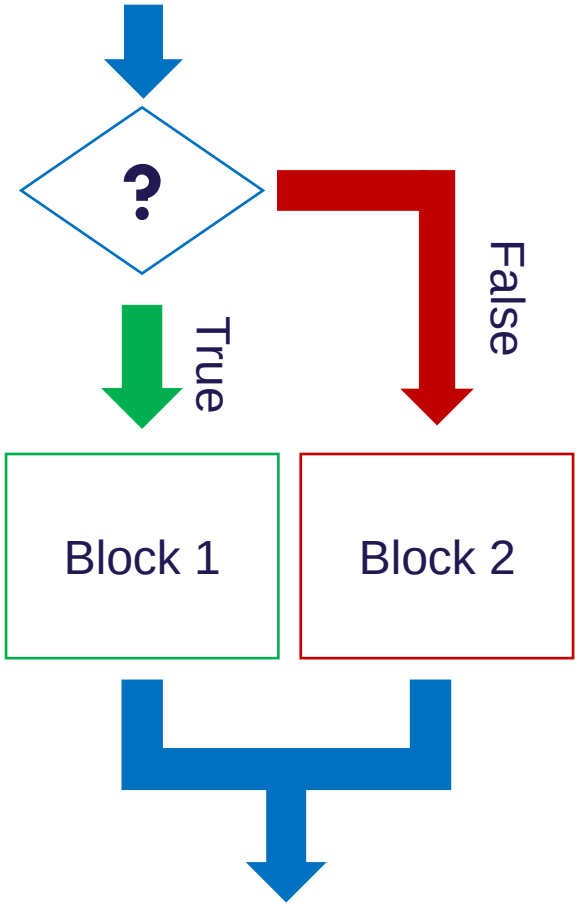
- Python's version of selecting one of two possible blocks
  - Allows you to handle a false condition with its own code block
  - In natural language

    If the condition is true then do block 1 else do block 2



The if-else statement

```
code before
if condition:
        do block 1
else:
        do block 2
code after
```



Example: if-else

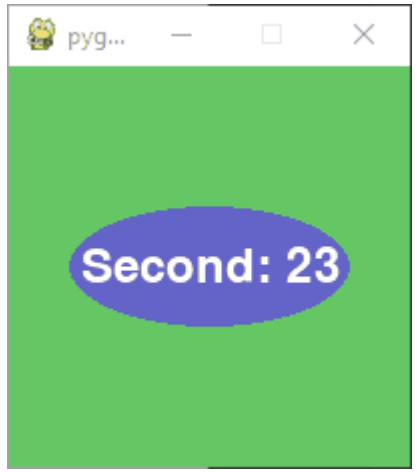
```
import pygame
import math
from datetime import datetime
pygame.init()
screen_size = (200, 200)
screen = pygame.display.set_mode(screen_size)
screen_center = (screen_size[0] // 2, screen_size[1] // 2)
font = pygame.font.SysFont(None, 36)

while True:
    screen.fill((100, 200, 100))
    s = datetime.now().second
    if s > 30:
        pygame.draw.rect(screen, (200, 100, 100), (30, 70, 140, 60))
    else:
        pygame.draw.ellipse(screen, (100, 100, 200), (30, 70, 140, 60))

    text = font.render(f'Second: {s}', True, (255, 255, 255))
    text_pos = (screen_center[0] - text.get_width() // 2,
                screen_center[1] - text.get_height() // 2)
    screen.blit(text, text_pos)

    pygame.display.flip()

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            exit()
```



Example: if-else

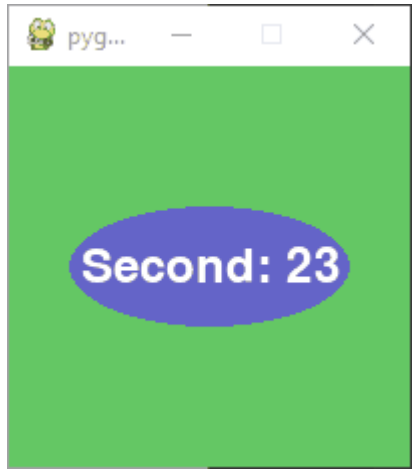
```
import pygame
import math
from datetime import datetime
pygame.init()
screen_size = (200, 200)
screen = pygame.display.set_mode(screen_size)
screen_center = (screen_size[0] // 2, screen_size[1] // 2)
font = pygame.font.SysFont(None, 36)

while True:
    screen.fill((100, 200, 100))
    s = datetime.now().second
    if s > 30:
        pygame.draw.rect(screen, (200, 100, 100), (30, 70, 140, 60))
    else:
        pygame.draw.ellipse(screen, (100, 100, 200), (30, 70, 140, 60))

    text = font.render(f'Second: {s}', True, (255, 255, 255))
    text_pos = (screen_center[0] - text.get_width() // 2,
                screen_center[1] - text.get_height() // 2)
    screen.blit(text, text_pos)

    pygame.display.flip()

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            exit()
```



Agenda

- Animation and a while sneak peek
- If statements
- **Conditionals and boolean expressions**
- Chaining and nesting



Logical expressions

- Anything that you can determine to be true or false
  - I.e. anything that produces a Boolean value

- The possible options
  - A Boolean value, `True` or `False`
  - A variable of type `bool`
  - Any expression evaluating to a value of type `bool`

} Probably the most interesting options



AALBORG UNIVERSITY
DENMARK

Logical expressions

```
if True:
    # Do something
    pass
```

```
b = True
# ... Some code ...
if b:
    # Do something
    pass
```

```
a = 10
# ... Some code ...
if a < 100:
    # Do something
    pass
```



Logical expressions

- Logical expressions are often made using two kinds of operators
  - Relational operators ← Compares/relates the values of two variables
  - Logical operators ← Combines several logical values into one
- The result is always either true or false!

Relational operators

- Equal to
- Not equal to
- Greater than
- Less than
- Greater than or equal to
- Less than or equal to
- `in`

```
my_list = [1, 2, 3, 4, 5]
2 in my_list
```

```
a, b = 1, 2

a == b

a != b

a > b

a < b

a >= b

a <= b
```

NOTE: == is not the same as =



Logical operators

- NOT
  - Inverts the value
- AND
  - *Both* operands must be true
- OR
  - *At least* one operand must be true

```
c, d = True, False

not c

c and d

c or d
```



Logical expressions

```python
if i < 10 and k > 75:
    # Do something
    pass

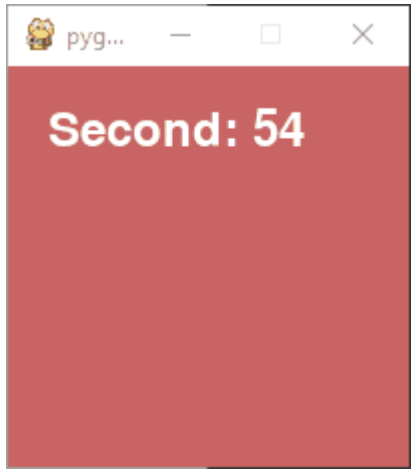
if i < 10 or k > 75:
    # Do something
    pass

if (a < 10 and b < 20 and c < 30) or q > 100:
    # Do something
    pass
```



Example: Compound logical expression

```
color_change_enabled = True
while True:
    s = datetime.now().second
    screen.fill((100, 200, 100))
    if color_change_enabled and s > 30:
        screen.fill((200, 100, 100))
```



Agenda

- Animation and a while sneak peek
- If statements
- Conditionals and boolean expressions
- **Chaining and nesting**



Nesting and chaining

- What can you put into the code blocks?
- Any code you like!
  - More `if` statements
  - Loops
  - Function calls

- Putting code blocks within code blocks is called *nesting*

Nesting

```
if a > 5:
    if a <= 10:
        # Do something
        pass
```

```
if a > 5 and a <= 10:
        # Do something
        pass
```

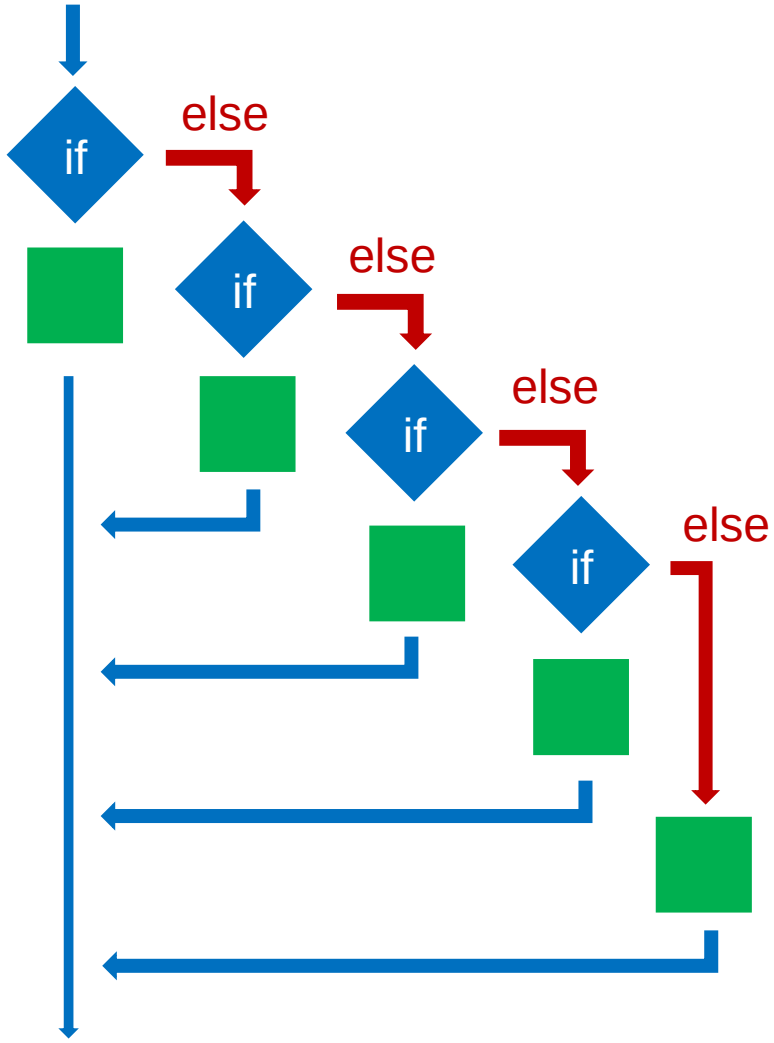
What's the difference?



AALBORG UNIVERSITY
DENMARK

Chaining

```
if age < 5:
    # show DR Ramasjang
    pass
elif age < 15:
    # show DR Ultra
    pass
elif age < 25:
    # show DR 3
    pass
else:
    # show DR 2
    pass
```



Chaining

```
if age < 5:
    # show DR Ramasjang
    pass
elif age < 15:
    # show DR Ultra
    pass
elif age < 25:
    # show DR 3
    pass
else:
    # show DR 2
    pass
```

```
if age < 5:
    # show DR Ramasjang
    pass
if age < 15:
    # show DR Ultra
    pass
if age < 25:
    # show DR 3
    pass
else:
    # show DR 2
    pass
```

What's the difference?



Exercise time!